Light

GCSE Computer Science - Programming Fundamentals (Set 1)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • AQA 8525 • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset ← Back to Programming Fundamentals

Multiple-choice topic test for **GCSE Computer Science (AQA 8525)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

What is a variable in programming?

- ○ **A.** A named location in memory that stores a value which can change during program execution
- ○ **B.** A fixed value that cannot be changed once assigned
- ○ **C.** A type of loop that repeats code
- ○ **D.** A section of code that performs a specific task

Q2 (1 mark)

What is a constant in programming?

- ○ **A.** A value that changes every time the program runs
- ○ **B.** A type of error that occurs during compilation
- ○ **C.** A temporary storage area that is deleted after use
- ○ **D.** A named value that is set once when declared and cannot be changed during program execution

Q3 (1 mark)

What is iteration in programming?

- ○ **A.** A way to make decisions in code based on conditions
- ○ **B.** The process of storing data in variables
- ○ **C.** The repetition of a block of code either a set number of times or until a condition is met
- ○ **D.** A method of displaying output to the user

Q4 (1 mark)

What is selection in programming?

- ○ **A.** A programming construct that allows different code paths to be executed based on whether a condition is true or false
- ○ **B.** A method for storing multiple values
- ○ **C.** A way to organise code into reusable blocks

- ○ **D.** The process of repeating code multiple times

Q5 (1 mark)

Which data type should be used to store a person's age in whole years?

- ○ **A.** Real
- ○ **B.** Integer
- ○ **C.** String
- ○ **D.** Boolean

Q6 (1 mark)

Which data type can only store the values `True` or `False`?

- ○ **A.** String
- ○ **B.** Boolean
- ○ **C.** Integer
- ○ **D.** Character

Q7 (1 mark)

What does the MOD operator return?

- ○ **A.** The remainder when one integer is divided by another

- ○ **B.** The whole number result of dividing one integer by another
- ○ **C.** The result of multiplying two numbers
- ○ **D.** The average of two numbers

Q8 (1 mark)

What does the DIV operator return?

- ○ **A.** The decimal result of dividing two numbers
- ○ **B.** The remainder after division
- ○ **C.** The sum of two integers
- ○ **D.** The whole number (integer) result of dividing one integer by another, discarding any remainder

Q9 (1 mark)

What is a string data type used for?

- ○ **A.** Storing only True or False values
- ○ **B.** Storing only numerical values
- ○ **C.** Storing sequences of characters such as words, sentences, or text
- ○ **D.** Storing only single characters

Q10 (1 mark)

What is concatenation?

- ○ **A.** Splitting a string into smaller parts
- ○ **B.** Finding the length of a string
- ○ **C.** Converting a string to uppercase
- ○ **D.** Joining two or more strings together to form a single string

Q11 (1 mark)

What is a subroutine?

- ○ **A.** A type of variable that stores multiple values
- ○ **B.** An error that occurs during program execution
- ○ **C.** A named block of code designed to perform a specific task that can be called from other parts of the program
- ○ **D.** A loop that runs indefinitely

Q12 (1 mark)

What is a local variable?

- ○ **A.** A variable that can be accessed from anywhere in the program
- ○ **B.** A variable that stores geographical coordinates

- ○ **C.** A variable that is declared inside a subroutine, exists only while that subroutine is executing, and can only be accessed within that subroutine
- ○ **D.** A variable that stores the program's location on disk

Q13 (2 marks)

What is the value of `result` after the following code executes?

```
x <- 5
y <- 3
x <- x + y
result <- x * 2
```

- ○ **A.** 16
- ○ **B.** 10
- ○ **C.** 13
- ○ **D.** 8

Q14 (2 marks)

What is output by the following code when `temperature` is 25?

```
temperature <- 25
IF temperature > 30 THEN
    OUTPUT "Hot"
ELSE
    IF temperature > 20 THEN
        OUTPUT "Warm"
    ELSE
        OUTPUT "Cool"
```

```
    ENDIF
ENDIF
```

- A. Hot
- B. Cool
- C. Warm
- D. Nothing is output

Q15 (2 marks)

What is the value of `total` after the following code executes?

```
total <- 0
FOR i <- 1 TO 4
    total <- total + i
ENDFOR
```

- A. 4
- B. 6
- C. 15
- D. 10

Q16 (2 marks)

A program needs to store the price of items in a shop, such as 4.99 or 12.50. Which data type is most appropriate?

- A. Boolean

- B. Integer
- C. String
- D. Real

Q17 (2 marks)

Given `word <- "COMPUTER"`, what is the result of `LENGTH(word)`?

- A. 8
- B. 7
- C. 9
- D. 6

Q18 (2 marks)

What are the values of **a** and **b** after the following code executes?

```
number <- 17
a <- number DIV 5
b <- number MOD 5
```

- A. a = 3.4, b = 0
- B. a = 3, b = 2
- C. a = 2, b = 3
- D. a = 5, b = 12

Q19 (2 marks)

Given `x <- 5` and `y <- 10`, what is the result of the Boolean expression `(x > 3) AND (y < 8)`?

- A. False
- B. 5
- C. True
- D. Error

Q20 (2 marks)

Given `a <- True` and `b <- False`, what is the result of `NOT(a OR b)`?

- A. True
- B. False
- C. b
- D. a

Q21 (2 marks)

What is the value of `count` when the following loop terminates?

```
count <- 1
WHILE count < 10
    count <- count + 3
ENDWHILE
```

- A. 7

- B. 9
- C. 12
- D. 10

Q22 (2 marks)

What is the purpose of the following code?

```
REPEAT
    OUTPUT "Enter a number between 1 and 100"
    number <- USERINPUT
UNTIL number >= 1 AND number <= 100
```

- A. To display numbers from 1 to 100
- B. To validate that user input is within the acceptable range of 1 to 100, re-prompting until valid input is received
- C. To add all numbers between 1 and 100
- D. To convert user input to a different data type

Q23 (2 marks)

What is output by the following code?

```
SUBROUTINE greet(name)
    OUTPUT "Hello, " + name
ENDSUBROUTINE

greet("Sam")
```

- A. Hello, name

- B. Sam
- C. Hello, Sam
- D. greet("Sam")

Q24 (3 marks)

What are the final values of **x**, **y**, and **z** after the following code executes?

```
x <- 10
y <- 3
z <- x MOD y
x <- x DIV y
y <- z + x
z <- y * 2
```

- **A.** x = 3, y = 4, z = 8
- **B.** x = 1, y = 4, z = 8
- **C.** x = 3, y = 1, z = 2
- **D.** x = 10, y = 4, z = 8

Q25 (3 marks)

What is output by the following code?

```
FOR i <- 1 TO 2
    FOR j <- 1 TO 3
        OUTPUT i * j
    ENDFOR
ENDFOR
```

- A. 1, 2, 3, 4, 5, 6
- B. 1, 2, 2, 4, 3, 6
- C. 1, 2, 3, 2, 4, 6
- D. 2, 4, 6, 2, 4, 6

Q26 (3 marks)

What is output by the following code?

```
FUNCTION double(n)
    RETURN n * 2
ENDFUNCTION

FUNCTION addFive(n)
    RETURN n + 5
ENDFUNCTION

result <- addFive(double(3))
OUTPUT result
```

- A. 6
- B. 11
- C. 16
- D. 8

Q27 (3 marks)

A validation routine checks that a password meets these requirements: at least 8 characters long AND contains at least one digit. Which password would pass this validation?

- A. Pass123
- B. Secret
- C. 12345678
- D. Password1

Q28 (3 marks)

What is output by the following code?

```
numbers <- [4, 7, 2, 9]
total <- 0
FOR i <- 0 TO 3
    IF numbers[i] > 5 THEN
        total <- total + numbers[i]
    ENDIF
ENDFOR
OUTPUT total
```

- A. 22
- B. 9
- C. 16
- D. 11

Q29 (3 marks)

A simple authentication system stores `correctUsername <- "admin"` and `correctPassword <- "secret123"`. A user attempts to log in with username "Admin" and password "secret123". Analyse why the login might fail and what this reveals about authentication design.

- ○ **A.** The login will always succeed because the password is correct
- ○ **B.** The login may fail because string comparison is typically case-sensitive: 'Admin' does not equal 'admin'. This reveals that authentication systems must clearly communicate case-sensitivity to users, or convert inputs to a consistent case before comparison
- ○ **C.** The login will fail because passwords should be longer
- ○ **D.** The login will succeed because usernames are not important

Q30 (3 marks)

A programmer is writing a quiz program. Instead of writing all code in one block, they create separate subroutines: `displayQuestion()`, `getAnswer()`, `checkAnswer()`, and `updateScore()`. Analyse the advantages of this structured approach.

- ○ **A.** This structured approach offers multiple advantages: code becomes easier to read and understand because each subroutine has a clear single purpose; debugging is simpler because errors can be isolated to specific subroutines; code can be reused (displayQuestion could show any question); different programmers can work on different subroutines simultaneously; and changes to one subroutine do not affect others if interfaces are maintained
- ○ **B.** This approach makes the program run faster
- ○ **C.** The only advantage is that the program uses less memory
- ○ **D.** There are no real advantages to using subroutines

Total: **55** marks • Time: **30 min**

↑ Back to top

Submit answers

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.